

Decoding Mushroom Toxicity with Logistic Regression Analysis

Submitted By:

Shams E Alam Prianka
Student ID: 2406876

Introduction

This project aims to analyze the Mushroom Dataset (UCI) using Logistic Regression with Gradient Descent. Both a custom (scratch) implementation using gradient descent and sci-kit learn Logistic Regression Model were applied. The dataset consists of 22 categorical features describing various characteristics of mushrooms and a target variable indicating whether the mushroom is non-toxic or toxic.

1. Dataset Overview

- Source: UCI Machine Learning Repository
- Source link: <https://archive.ics.uci.edu/dataset/73/mushroom>
- Features: 22 features (e.g., cap-shape, odor, gill color)
- Instances: 8124 instances with missing values
- Target Variable: Edibility (edible vs. poisonous)
- Motivation: The dataset is well-structured, has missing values, and is ideal for binary classification.

```
: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8124 entries, 0 to 8123
Data columns (total 23 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   poisonous                             8124 non-null   object
1   cap-shape                             8124 non-null   object
2   cap-surface                           8124 non-null   object
3   cap-color                             8124 non-null   object
4   bruises                              8124 non-null   object
5   odor                                  8124 non-null   object
6   gill-attachment                       8124 non-null   object
7   gill-spacing                          8124 non-null   object
8   gill-size                             8124 non-null   object
9   gill-color                            8124 non-null   object
10  stalk-shape                           8124 non-null   object
11  stalk-root                            5644 non-null   object
12  stalk-surface-above-ring              8124 non-null   object
13  stalk-surface-below-ring              8124 non-null   object
14  stalk-color-above-ring                8124 non-null   object
15  stalk-color-below-ring                8124 non-null   object
16  veil-type                             8124 non-null   object
17  veil-color                            8124 non-null   object
18  ring-number                           8124 non-null   object
19  ring-type                             8124 non-null   object
20  spore-print-color                     8124 non-null   object
21  population                            8124 non-null   object
22  habitat                               8124 non-null   object
dtypes: object(23)
memory usage: 1.4+ MB
```

Interpretation: This justifies the need for label encoding and missing value handling. It is a clear representation that no numerical columns exist, thus one-hot encoding or label encoding is mandatory.

Convert categorical features into numerical representations:

Column	Categories	Numerical Categories
poisonous	p = poisonous e = edible	p = 1 e = 0
cap-shape	bell=b,conical=c,convex=x,flat=f, knobbed=k,sunken=s	b = 0, c = 1, f = 2, k = 3, s = 4, x = 5

cap-surface	fibrous=f,grooves=g,scaly=y,smooth=s	f = 0, g = 1, s = 2, y = 3
cap-color	brown=n, buff=b, cinnamon=c, gray=g, green=r, pink=p, purple=u, red=e, white=w, yellow=y	b = 0, c = 1, e = 2, g = 3, n = 4, p = 5, r = 6, u = 7, w = 8, y = 9
bruises	bruises=t, no=f	f = 0, t = 1
odor	almond=a, anise=l, creosote=c, fishy=y, foul=f, musty=m, none=n, pungent=p, spicy=s	a = 0, c = 1, f = 2, l = 3, m = 4, n = 5, p = 6, s = 7, y = 8
gill-attachment	attached=a, descending=d, free=f, notched=n	a = 0, d = 1, f = 2, n = 3
gill-spacing	close=c, crowded=w, distant=d	c = 0, d = 1, w = 2
gill-size	broad=b, narrow=n	b = 0, n = 1
gill-color	black=k, brown=n, buff=b, chocolate=h, gray=g, green=r, orange=o, pink=p, purple=u, red=e, white=w, yellow=y	b = 0, e = 1, g = 2, h = 3, k = 4, n = 5, o = 6, p = 7, r = 8, u = 9, w = 10, y = 11
stalk-shape	enlarging=e, tapering=t	e = 0, t = 1
stalk-root	bulbous=b, club=c, cup=u, equal=e, rhizomorph=hs=z, rooted=r, missing=?	b = 0, c = 1, e = 2, r = 3, u = 4, ? = ?, z = 5
stalk-surface-above-ring	fibrous=f, scaly=y, silky=k, smooth=s	f = 0, k = 1, s = 2, y = 3
stalk-surface-below-ring	fibrous=f, scaly=y, silky=k, smooth=s	f = 0, k = 1, s = 2, y = 3
stalk-color-above-ring	brown=n, buff=b, cinnamon=c, gray=g, orange=o, pink=p, red=e, white=w, yellow=y	b = 0, c = 1, e = 2, g = 3, n = 4, o = 5, p = 6, w = 7, y = 8
stalk-color-below-ring	brown=n, buff=b, cinnamon=c, gray=g, orange=o, pink=p, red=e, white=w, yellow=y	b = 0, c = 1, e = 2, g = 3, n = 4, o = 5, p = 6, w = 7, y = 8
veil-type	partial=p, universal=u	p = 0, u = 1
veil-color	brown=n, orange=o, white=w, yellow=y	n = 0, o = 1, w = 2, y = 3
ring-number	none=n, one=o, two=t	n = 0, o = 1, t = 2
ring-type	cobwebby=c, evanescent=e, flaring=f, large=l, none=n, pendant=p, sheathing=s, zone=z	c = 0, e = 1, f = 2, l = 3, n = 4, p = 5, s = 6, z = 7
spore-print-color	black=k, brown=n, buff=b, chocolate=h, green=r, orange=o, purple=u, white=w, yellow=y	b = 0, h = 1, k = 2, n = 3, o = 4, r = 5, u = 6, w = 7, y = 8
population	abundant=a, clustered=c, numerous=n, scattered=s, several=v, solitary=y	a = 0, c = 1, n = 2, s = 3, v = 4, y = 5
habitat	grasses=g, leaves=l, meadows=m, paths=p, urban=u, waste=w, woods=d	d = 0, g = 1, l = 2, m = 3, p = 4, u = 5, w = 6

2. Data Preprocessing

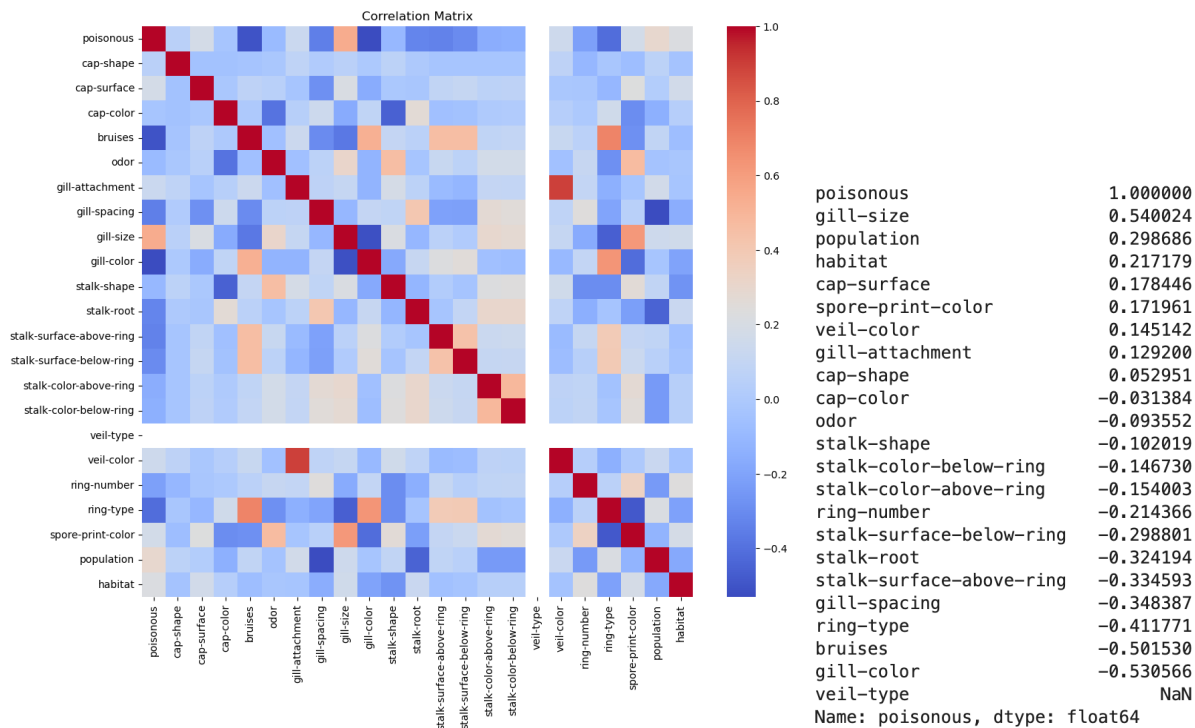
Data cleaning: The ‘stalk-root’ column contains 2,480 missing values represented by ‘?’. To improve prediction accuracy, it's important to clean the dataset by replacing all ‘?’ entries with actual NaN values so that pandas can properly recognize them as missing. Below is a comparison of the column before and after handling the missing values.

```
df.isnull().sum()
```

```
poisonous      0
cap-shape      0
cap-surface    0
cap-color      0
bruises        0
odor           0
gill-attachment 0
gill-spacing   0
gill-size      0
gill-color     0
stalk-shape    0
stalk-root     2480
stalk-surface-above-ring 0
stalk-surface-below-ring 0
stalk-color-above-ring 0
stalk-color-below-ring 0
veil-type      0
veil-color     0
ring-number    0
ring-type      0
spore-print-color 0
population     0
habitat        0
dtype: int64
```

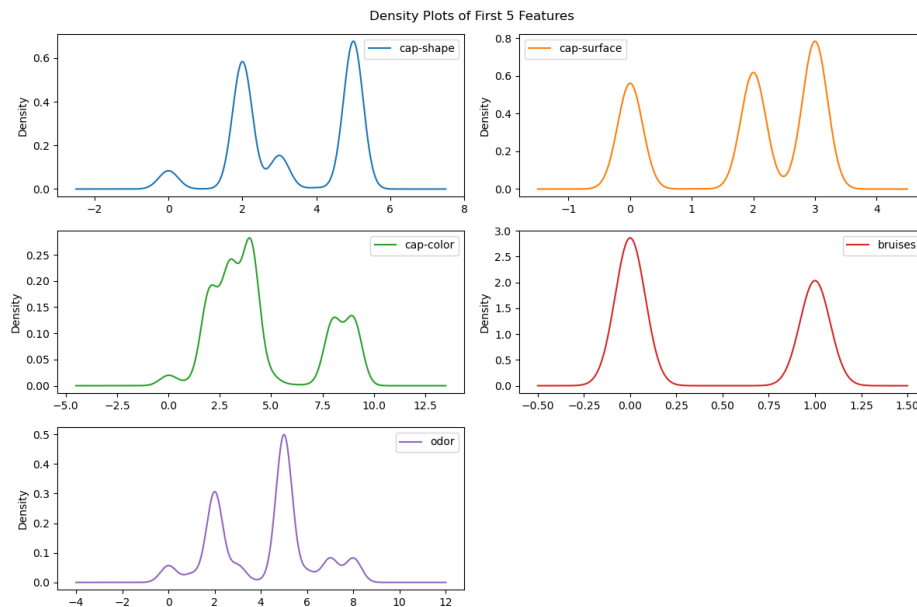
```
print(df.isna().sum())
```

```
poisonous      0
cap-shape      0
cap-surface    0
cap-color      0
bruises        0
odor           0
gill-attachment 0
gill-spacing   0
gill-size      0
gill-color     0
stalk-shape    0
stalk-root     0
stalk-surface-above-ring 0
stalk-surface-below-ring 0
stalk-color-above-ring 0
stalk-color-below-ring 0
veil-type      0
veil-color     0
ring-number    0
ring-type      0
spore-print-color 0
population     0
habitat        0
dtype: int64
```



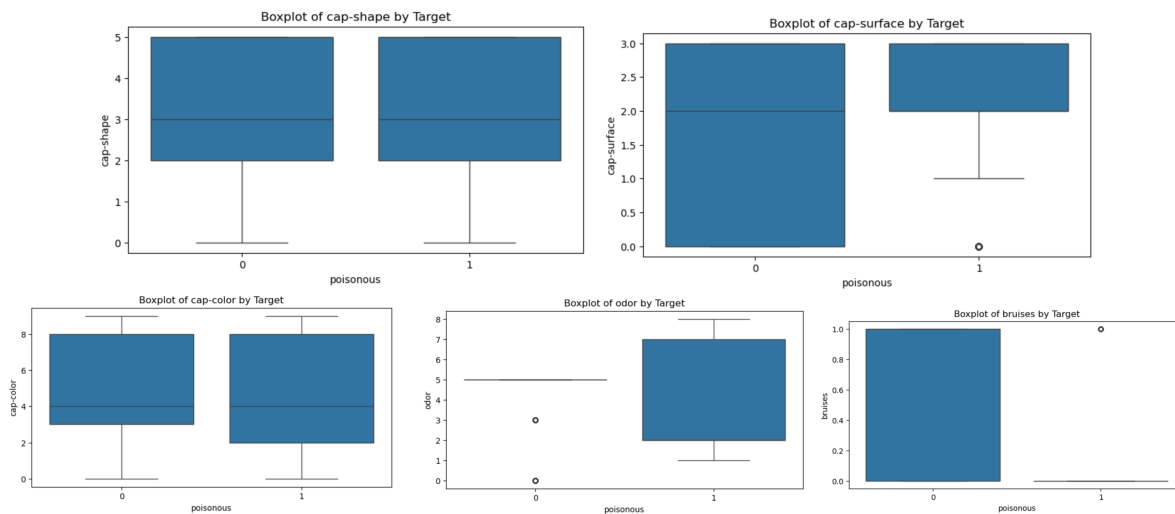
Interpretation: The correlation matrix and feature-to-target correlation table highlight the most predictive variables for identifying poisonous mushrooms. Features like gill-size, gill-color, and bruises show strong relationships with toxicity and thus are crucial in model training. While multicollinearity between features is generally low, these insights also support the interpretability of logistic regression in this binary classification problem. Finally, after examining the correlation between the feature variables, we are not planning to remove any variables moving forward.

Density plot of the features:



The density plots help visualize the distribution of encoded categorical variables. Features like ‘odor’ and ‘cap-color’ exhibit high variability and are likely influential in distinguishing between non-toxic and toxic mushrooms. Binary features like bruises show well-separated distributions that make them useful for classification. These plots also confirm that the label encoding process preserved meaningful distinctions in feature representation.

Boxplots:



3. Split dataset into training and testing sets:

The dataset was split into training and testing sets using an 80/20 split, resulting in 6,499 training samples and 1,625 testing samples. Each sample includes 22 categorical features after preprocessing. This stratified split ensures sufficient data for both model learning and validation, allowing for reliable performance evaluation on unseen examples.

```
x_train.shape
```

```
(6499, 22)
```

```
x_test.shape
```

```
(1625, 22)
```

```
x_train.shape
```

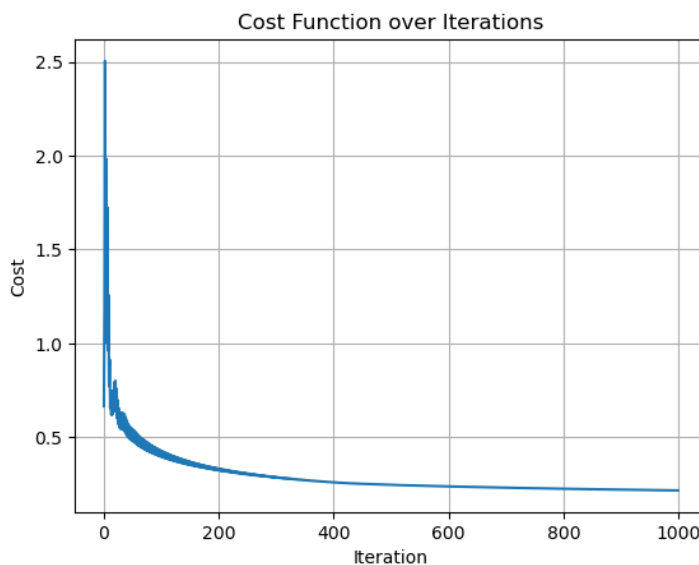
```
(6499, 22)
```

```
y_test.shape
```

```
(1625,)
```

4. Model Implementation:

I Implemented logistic regression from scratch using gradient descent. A learning rate sensitivity analysis shows that a higher learning rate (0.1) leads to better convergence in both accuracy and log-loss. At lower learning rates (e.g., 0.01), the model struggles to converge within the given number of iterations, resulting in lower accuracy and higher uncertainty. Thus, learning_rate = 0.1 was chosen for the final model training.



Feature Weights:

gill-size	2.578724
veil-color	1.037433
population	0.473372
cap-surface	0.469103
gill-attachment	0.375727
habitat	0.266160
cap-shape	0.037160
stalk-color-below-ring	0.030625
cap-color	0.013427
veil-type	0.000000
ring-type	-0.022715
stalk-color-above-ring	-0.054094
spore-print-color	-0.083653
gill-color	-0.159237
odor	-0.167523
stalk-surface-below-ring	-0.348081
ring-number	-0.617421
stalk-shape	-0.902329
stalk-root	-0.975589
bruises	-0.986027
stalk-surface-above-ring	-1.249071
gill-spacing	-1.327167
dtype:	float64

Model Configuration:

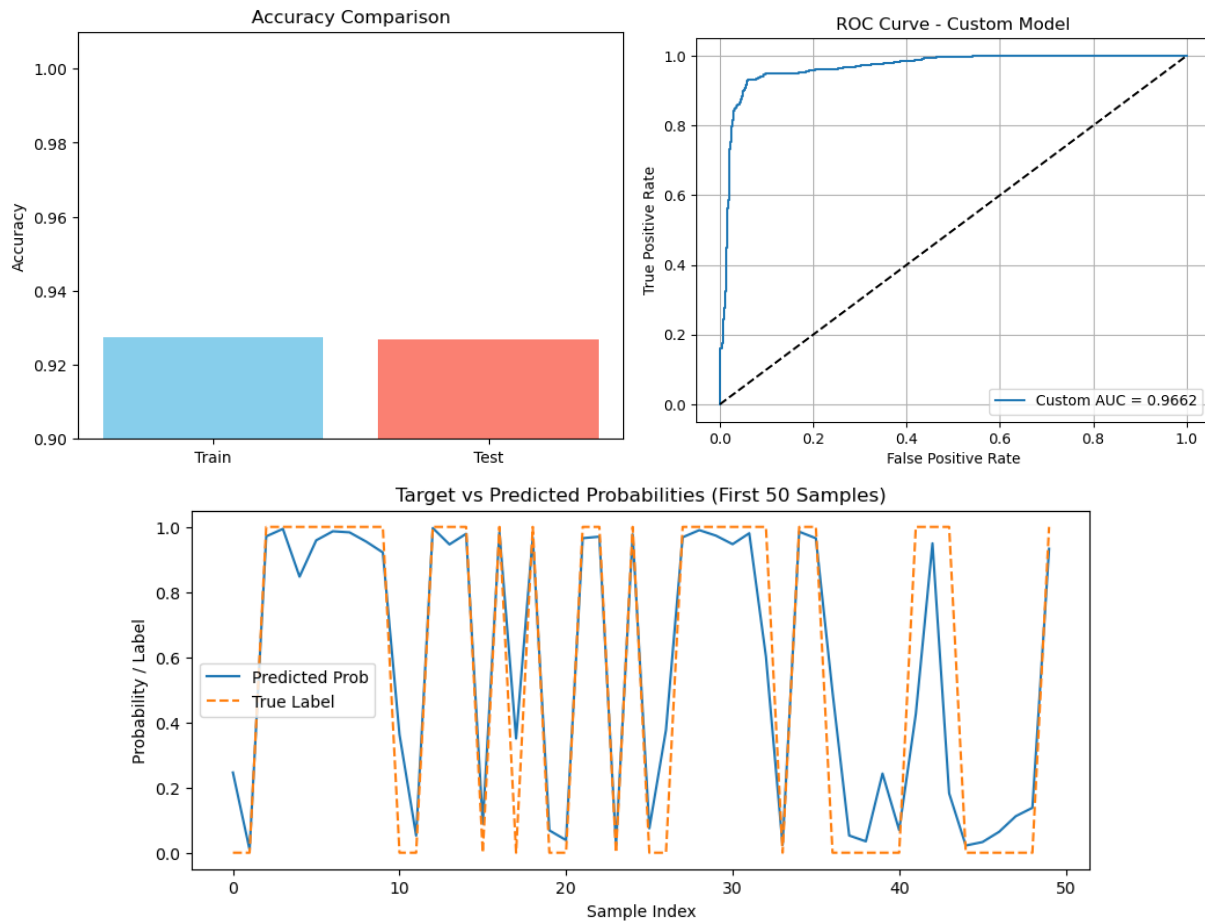
- Learning rate: 0.01
- Max Iterations: 1000
- Convergence Tolerance: 1e-6
- Sigmoid activation and bias term included.

Custom model summary:

Train Accuracy: 0.8817

Test Accuracy: 0.8757

Train Log Loss: 0.3278
Test Log Loss: 0.3307



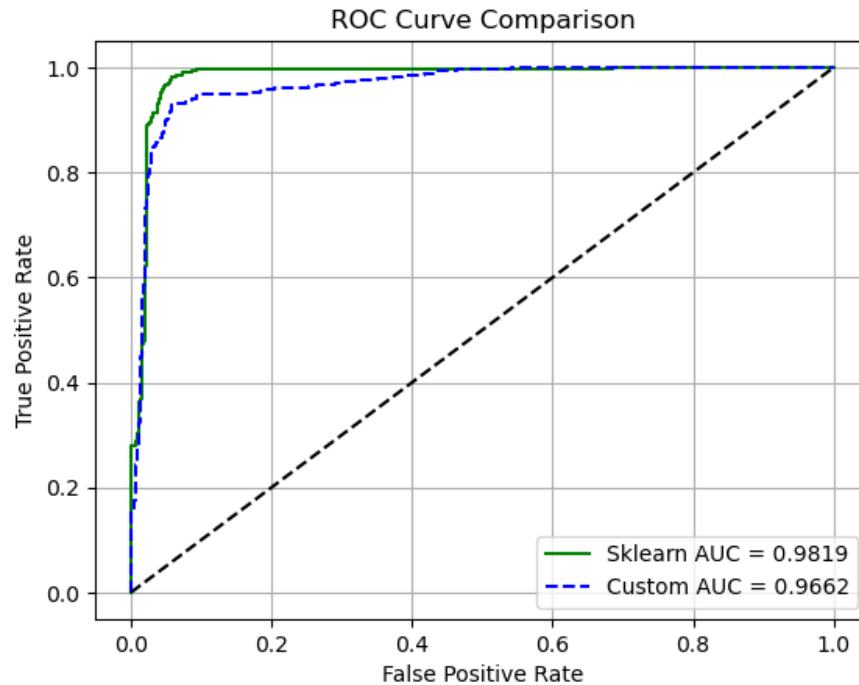
Experiment with different learning rates:

Learning rate: 0.01	Learning rate: 0.05	Learning rate: 0.1
Train Accuracy: 0.8817	Train Accuracy: 0.9188	Train Accuracy: 0.9275
Test Accuracy: 0.8757	Test Accuracy: 0.9126	Test Accuracy: 0.9268
Train Log Loss: 0.3278	Train Log Loss: 0.2484	Train Log Loss: 0.2170
Test Log Loss: 0.3307	Test Log Loss: 0.2538	Test Log Loss: 0.2259

5. Chosen Final model:

- Learning rate: 0.1
- Max Iterations: 1000
- Convergence Tolerance: 1e-6
- Sigmoid activation and bias term included.

Custom model summary	Scikit-Learn Model Evaluation
Train Accuracy: 0.9275	Train Accuracy: 0.9598
Test Accuracy: 0.9268	Test Accuracy: 0.9545
Train Log Loss: 0.2170	Train Log Loss: 0.1413
Test Log Loss: 0.2259	Test Log Loss: 0.1589



The ROC curve comparison highlights the excellent classification ability of both models. The Scikit-learn model achieved an AUC of 0.9819, while the custom implementation reached 0.9662. The curves indicate strong true positive rates and low false positive rates across various thresholds, confirming that both models are highly capable of separating edible from poisonous mushrooms. This visual metric reinforces the reliability and effectiveness of logistic regression for this binary classification task.

6. Why are these results too high?

- The UCI Mushroom dataset is clean, contains only categorical variables, and has minimal noise. Many variables have clear patterns that separate edible and toxic mushrooms.
- Some odors only appear in toxic mushrooms, while others are exclusive to non-toxic ones. This allows even simple models to achieve high accuracy.
- There are no mislabeled data points or overlap between classes, making it easy for the model to learn correct associations.
- Scikit-learn's Logistic Regression applies L2 regularization, which helps it generalize slightly better and achieve lower log loss.
- Missing values were handled, features were properly encoded, and learning rate and convergence settings were tuned. All of this improves training stability and performance.

Insights & Conclusions

The logistic regression models applied to the UCI Mushroom dataset delivered highly accurate classification results. The custom model achieved a test accuracy of 92.68% with a log loss of 0.2259, while Scikit-learn's model performed slightly better, reaching 95.45% test accuracy and 0.1589 test log loss.

These high scores are expected given the dataset's structure, especially the strong predictive power of features like 'odor' and 'gill-size'. The learning rate experiment confirmed that 0.1 provided the best convergence, outperforming lower rates such as 0.01, which resulted in only 87.57% accuracy. Also, the outliers did not significantly affect the prediction model.

There was no indication of overfitting, as training and test metrics remained close for both models. All 22 features were retained, and proper preprocessing, including handling missing values and encoding categorical variables, contributed to reliable and interpretable outcomes.

This project validates logistic regression as a simple yet powerful approach for binary classification on clean, categorical datasets. The custom implementation closely matched the Scikit-learn model, confirming both its correctness and effectiveness.